

Universidad Sergio Arboleda

Inteligencia Artificial

Optimización por Colonia de Hormigas

Taller de optimización combinatoria: el Agente Viajero

Integrantes:

Lina Castañeda

Jorge García

Brayan Hernandez

Docente:

Joaquin Sánchez

Bogotá D.C.

2026

Índice

1. Introducción	3
2. Descripción del problema	3
3. Punto 1. Formulación del problema	3
3.1. Variable de decisión	3
3.2. Función objetivo	3
3.3. Número de soluciones diferentes	4
3.4. Cálculo manual con un ejemplo pequeño	4
4. Punto 2. Diseño del algoritmo ACO	4
4.1. Variante y parámetros	4
4.2. Construcción de soluciones	5
4.3. Actualización de feromona	5
4.4. Mejora local 2-opt	5
4.5. Modos de ejecución	5
4.6. Complejidad	5
5. Punto 3. Implementación y comandos de ejecución	6
5.1. Estructura del repositorio	6
5.2. Compilación	6
5.3. Comandos de ejecución	6
5.4. Evidencia de ejecución	7
6. Punto 4. Evaluación experimental	7
6.1. Resultados	7
6.2. Análisis de resultados	7
6.3. Conclusión de la evaluación experimental	8

1. Introducción

La optimización por colonia de hormigas (*Ant Colony Optimization*, ACO) es una metaheurística inspirada en el comportamiento de las hormigas reales para encontrar caminos cortos entre su nido y las fuentes de alimento. Las hormigas artificiales construyen soluciones de forma probabilística guiadas por rastros de feromona y por información heurística del problema.

En este taller se diseñó e implementó un algoritmo ACO para resolver el problema del Agente Viajero (*Travelling Salesman Problem*, TSP). El objetivo no es solamente encontrar tours cortos, sino medir el desempeño del algoritmo en **tiempo** y en **memoria** para tres tamaños de instancia: 20, 2000 y 200000 ciudades, ejecutando todo en local.

La implementación se realizó en C++17 con paralelismo OpenMP y se publicó en el repositorio github.com/JAGR1792/ACO.

2. Descripción del problema

Se dispone de un conjunto de n ciudades con coordenadas en el plano. Se debe encontrar el recorrido cerrado más corto que visite cada ciudad exactamente una vez y regrese a la ciudad de origen.

Las instancias son euclidianas con puntos uniformes aleatorios en $[0, 1000]^2$, generadas con semilla fija (`-seed 42`) para que sean reproducibles. Se evaluaron tres tamaños:

Instancia	Ciudades	Semilla
Pequeña	20	42
Mediana	2000	42
Grande	200000	42

3. Punto 1. Formulación del problema

3.1. Variable de decisión

Una solución se representa como una permutación $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ de las n ciudades, que indica el orden de visita. El tour es cerrado: después de π_n se regresa a π_1 .

3.2. Función objetivo

Se minimiza la distancia total del recorrido:

$$\text{mín } L(\pi) = \sum_{i=1}^{n-1} d(\pi_i, \pi_{i+1}) + d(\pi_n, \pi_1) \quad (1)$$

donde $d(a, b)$ es la distancia euclidiana entre las ciudades a y b .

3.3. Número de soluciones diferentes

Fijando la ciudad inicial y sin distinguir el sentido del recorrido, el número de tours diferentes es:

$$\frac{(n-1)!}{2} \quad (2)$$

Cuadro 1: Tamaño del espacio de búsqueda.

n	Tours diferentes
20	$\approx 6,1 \times 10^{16}$
2000	del orden de 10^{5735}
200000	intratable por enumeración

Por lo tanto, la enumeración exhaustiva solo es concebible para valores muy pequeños de n , lo que justifica el uso de una metaheurística como ACO.

3.4. Cálculo manual con un ejemplo pequeño

Con 4 ciudades $A(0,0)$, $B(3,0)$, $C(3,4)$, $D(0,4)$ existen $(4-1)!/2 = 3$ tours diferentes (fijando A como inicio):

Tour	Longitud
$A-B-C-D-A$	$3 + 4 + 3 + 4 = 14$
$A-B-D-C-A$	$3 + 5 + 3 + 5 = 16$
$A-C-B-D-A$	$5 + 4 + 5 + 4 = 18$

El óptimo es el tour $A-B-C-D-A$ con longitud 14. Este ejemplo muestra que la función objetivo es simplemente la suma de las aristas del recorrido cerrado.

4. Punto 2. Diseño del algoritmo ACO

4.1. Variante y parámetros

Se implementó la variante **MMAS** (*Max-Min Ant System*) simplificada con mejora local 2-opt. Sus parámetros fueron:

Cuadro 2: Configuración del algoritmo ACO.

Parámetro	Símbolo	Valor
Hormigas por iteración	m	20 (n=20), 25 (n=2000)
Iteraciones	$t_{\text{máx}}$	100 (n=20), 50 (n=2000)
Peso de feromona	α	1,0
Peso heurístico	β	2,5
Evaporación	ρ	0,1
Candidatos k-NN	k	8 (n=20), 25 (n=2000)

4.2. Construcción de soluciones

Cada hormiga construye un tour paso a paso. Estando en la ciudad i , la probabilidad de moverse a la ciudad no visitada j es proporcional a $\tau_{ij}^\alpha \cdot \eta_{ij}^\beta$, donde τ_{ij} es la feromona y $\eta_{ij} = 1/d_{ij}$ la información heurística.

Para no evaluar las n ciudades en cada paso se usa una **lista de candidatos**: solo se consideran las k vecinas más cercanas no visitadas (si ya se visitaron todas, se usa el resto). Esto reduce el costo por paso de $O(n)$ a $O(k)$.

4.3. Actualización de feromona

En cada iteración se evapora toda la feromona con factor $(1 - \rho)$ y solo la **mejor hormiga global** deposita feromona ($1/L_{\text{mejor}}$) en las aristas de su tour, con límites $[\tau_{\text{mín}}, \tau_{\text{máx}}]$ para evitar la convergencia prematura.

4.4. Mejora local 2-opt

Al mejor tour de cada iteración (y al mejor final) se le aplica 2-opt *first-improvement*: se prueba a reconectar pares de aristas y se acepta el primer intercambio que acorte el tour (ventana de 40 vecinos si $n > 500$).

4.5. Modos de ejecución

Cuadro 3: Modos del programa.

Modo	Uso	Descripción
full	$n \leq 5000$	ACO denso: matriz $n \times n$ + k-NN + MMAS + 2-opt
hier	n grande	Jerárquico: rejilla $\rightarrow$ ACO por celda $\rightarrow$ unión por centroides
nn	referencia	Vecino más cercano greedy (+ 2-opt si $n \leq 5000$)
auto	defecto	full si $n \leq 5000$, hier en otro caso

El modo **hier** es necesario para 200000 ciudades: una matriz densa de 200000^2 valores en **float** ocuparía ≈ 149 GB **por matriz** (el programa lo reporta como **matrix_mb_est**), imposible en local. En su lugar se particiona el plano en una rejilla de celdas de ≈ 600 puntos, se corre un ACO pequeño e independiente por celda y se concatenan los subtours ordenando las celdas por centroide.

4.6. Complejidad

- Memoria en modo **full**: $O(n^2)$ (matriz de distancias + matriz de feromona).
- Tiempo en modo **full**: $O(t_{\text{máx}} \cdot m \cdot n \cdot k)$ con candidatos.
- Modo **hier**: $O(n)$ en memoria global + ACO sobre celdas de tamaño acotado.

Las hormigas de cada iteración se construyen en paralelo con OpenMP.

5. Punto 3. Implementación y comandos de ejecución

5.1. Estructura del repositorio

- `src/tsp.h`: instancia euclidiana, matriz densa, k-NN exacto.
- `src/aco.h`: solver MMAS + 2-opt.
- `src/main.cpp`: CLI, modos, particionado en rejilla, salida CSV.
- `src/metrics.h`: cronómetro (`chrono`) y pico de RAM (`getrusage`).
- `CMakeLists.txt`: compilación Release -O3 con OpenMP.
- `scripts/run_benchmark.sh`: corre los 3 casos y genera `results.csv`.
- `scripts/plot.py`: genera `benchmark.png`.

5.2. Compilación

Vía CMake (recomendada):

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
```

Vía manual, sin cmake (un solo comando):

```
g++ -O3 -std=c++17 -fopenmp src/main.cpp -o aco
```

5.3. Comandos de ejecución

```
# 20 ciudades: ACO completo
./build/aco --n 20 --ants 20 --iters 100 --cand 8 \
--seed 42 --mode full

# 2000 ciudades: ACO completo con 25 candidatos
./build/aco --n 2000 --ants 25 --iters 50 --cand 25 \
--seed 42 --mode full --out results.csv

# 200000 ciudades: ACO jerárquico (celdas de ~600 puntos)
./build/aco --n 200000 --seed 42 --mode hier \
--target-cluster 600 --out results.csv

# Benchmark completo + gráfica
bash scripts/run_benchmark.sh
python3 scripts/plot.py results.csv
```

Parámetros: `-n` ciudades, `-ants` hormigas por iteración, `-iters` iteraciones, `-cand` vecinos candidatos (0 = sin lista), `-seed` semilla, `-target-cluster` tamaño de celda en modo hier, `-out` archivo CSV, `-no-2opt` desactiva la mejora local.

5.4. Evidencia de ejecución

Figura 1: Salidas reales del programa en local.

```
n=20 mode=full ants=20 iters=100 seed=42 length=4460.31
time_ms=142.6 peak_kb=4452
matrix_mb_est=0.00 (solo modo full)

n=2000 mode=full ants=25 iters=50 seed=42 length=38698.88
time_ms=1218.2 peak_kb=52028
matrix_mb_est=15.26 (solo modo full)

[hier] n=200000 G=19 clusters=361
n=200000 mode=hier ants=20 iters=100 seed=42 length=408901.41
time_ms=6988.3 peak_kb=74796
matrix_mb_est=152587.89 (solo modo full)
```

6. Punto 4. Evaluación experimental

6.1. Resultados

Máquina: Arch Linux, 12 núcleos, 14 GB RAM, g++ 16.2.1, OpenMP. Semilla 42 en los tres casos.

Cuadro 4: Desempeño en tiempo y memoria.

n	Modo	Hormigas $\times$ iters	Longitud	Tiempo	Memoria pico
20	full (k=8)	20×100	4460,31	0,14 s	4,4 MB
2000	full (k=25)	25×50	38698,88	1,22 s	52,0 MB
200000	hier (361 celdas)	10×25 por celda	408901,41	6,99 s	75,0 MB

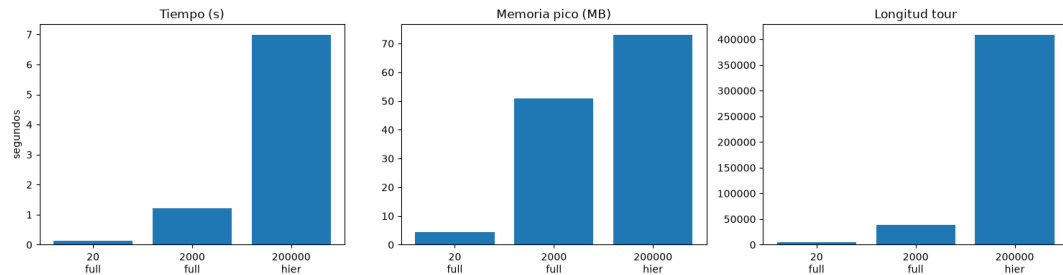


Figura 2: Tiempo, memoria pico y longitud del tour para los tres tamaños.

6.2. Análisis de resultados

a) **¿Cómo escala el tiempo?** De 20 a 2000 ciudades ($\times 100$ en n) el tiempo solo sube de 0,14 s a 1,22 s ($\times 9$), gracias a la lista de candidatos: cada paso cuesta $O(k)$ en vez de $O(n)$. El caso de 200000 se resuelve en ≈ 7 s porque el trabajo se divide en 361 ACOs de ≈ 550 puntos (costo casi lineal en n) más el ordenado de

centroides $O(C^2)$ con $C = 361$, despreciable. Sin el enfoque jerárquico, un ACO denso en 200000 ciudades sería intratable en tiempo ($O(t_{\text{máx}} \cdot m \cdot n^2)$ por iteración).

b) ¿Cómo escala la memoria? Con $n = 20$ se usan 4,4 MB (sobrecarga base del proceso y los hilos OpenMP; las matrices de 20×20 son insignificantes). Con $n = 2000$ se usan 52 MB: ≈ 15 MB de distancias + ≈ 15 MB de feromona + k-NN y tours de 25 hormigas, coherente con el $O(n^2)$ teórico. Con $n = 200000$ se usan solo 75 MB: nunca se construye la matriz global (que pediría ≈ 149 GB por matriz); cada celda aloja matrices de $\approx 600 \times 600$ ($\approx 1,4$ MB) y el resto es el vector de 200000 puntos. El diseño jerárquico cambia la memoria de $O(n^2)$ a $O(n)$.

c) ¿Qué aportan los candidatos y el 2-opt? Son los que más mejoran la relación calidad/tiempo: los candidatos reducen el paso de $O(n)$ a $O(k)$ y el 2-opt corrige cruces del tour construido por las hormigas. Desactivarlos (`-cand 0`, `-no-2opt`) empeora la longitud, el tiempo o ambos.

d) ¿Son comparables las longitudes entre tamaños? No directamente: cada n usa una instancia aleatoria distinta (más ciudades $\Rightarrow$ tour más largo). Para comparar variantes (distinto k , ρ o con/sin 2-opt) hay que fijar el mismo n y `-seed` y mirar la columna `length` del CSV.

e) ¿Qué configuración se recomienda? Para miles de ciudades, ACO denso con $k \approx 25$ y 2-opt activado (caso $n = 2000$: 1,2 s y 52 MB). Para cientos de miles, el esquema jerárquico con celdas de ≈ 600 puntos (≈ 7 s y 75 MB), aceptando que la unión greedy de subtours sacrifica optimalidad global. Trabajo futuro: orientar y coser subtours vecinos en vez de concatenar, y 2-opt entre celdas.

6.3. Conclusión de la evaluación experimental

1. El ACO clásico con matriz densa es viable hasta algunos miles de ciudades en local (2000 $\Rightarrow$ 1,2 s, 52 MB).
2. Para 200000 ciudades el ACO puro es inviable por memoria (≈ 149 GB por matriz); el esquema jerárquico lo hace posible en ≈ 7 s y 75 MB.
3. Con la misma semilla, la longitud del tour es reproducible en cualquier máquina; tiempo y memoria solo son comparables dentro de la misma máquina.